Werkproces uitwerking SD-Portfolio

Werkproces	K1W2: Maakt een technisch ontwerp voor software
Studentnaam	Yakub Bulut
Studentnummer	97118569
Klascode	SDA3
Datum oplevering	10/27/2026

Student verklaart hierbij dat de bewijsstukken in dit STARR-document

- Authentiek zijn
- Kwalitatief goed zijn
- Binnen een redelijke tijdsduur tot stand zijn gekomen

Bewijsstuk

Per bewijsstuk moet er gekeken worden naar hoe sterk deze is, hiervoor kijk je naar twee dingen:

1. De context – Waar en met wie heb je het gedaan?
2. Je zelfstandigheid– Hoe zelfstandig heb je gewerkt?

Voor zowel de context als de zelfstandigheid kan een score A (hoog), B (middelmatig) of C (laag) gegeven worden. Een bewijsstuk met de score ‘A - A’ (goede context én zelfstandig gewerkt) heeft een hogere waarde dan een bewijsstuk met de score ‘B - C’.

Maak voor het bepalen van de complexiteit gebruik van de "Context check" en de "Zelfstandigheid check”.

Korte beschrijving	Context	Zelfstandigheid
In deze bewijsstuk laat ik zien hoe ik de admin pagina heb gebouwd voor onze project op school XPtracker	Alleen Context check Uitkomst (B)	Context check Uitkomst (B)

STARR-reflectie

Situatie	Ik heb dit bewijsstuk op school gemaakt, in sprint 7 van ons schoolproject XP Tracker. XP Tracker is een website waar gamers hun statistieken van verschillende games op één plek kunnen bekijken. We werken in een team en in sprints, met Laravel als backend en Vue.js als frontend. In deze sprint moest ik een adminpaneel bouwen. Daarmee kan een admin gebruikers beheren (blokkeren, verwijderen, admin maken), games toevoegen of verbergen, cijfers bekijken op een dashboard en controleren of de externe API's werken. Het moest ook veilig zijn: alleen admins mogen erin. Daarom hebben we de rollen user, admin en owner gemaakt, en de controle zit in de backend.
Taak	In sprint 7 moest ik het adminpaneel van XP Tracker maken en de data eraan koppelen. Dit is maar een deel van het project, niet het hele project • Taak 1: in de database rollen toevoegen (user, admin en owner) en zorgen dat alleen admins in het adminpaneel kunnen. • Taak 2: de backend voor de admin maken en de data koppelen, zoals gebruikers, games, de cijfers op het dashboard en de status van de API's. • Taak 3: de adminpagina in de frontend maken, aan de backend koppelen en testen of alles werkt.
Activiteiten	Eerst heb ik naar de database gekeken. Ik heb een applicatiearchitectuur gemaakt om te zien welke tabellen de admin nodig heeft. Daarna heb ik met een migratie de kolommen role, banned_at en is_visible toegevoegd. Voor de rollen user, admin en owner heb ik een enum gemaakt. Daarna ben ik bezig geweest met de beveiliging. Ik heb twee middlewares gemaakt: één die ervoor zorgt dat alleen admins in het adminpaneel kunnen, en één die geblokkeerde gebruikers meteen uitlogt. Een geblokkeerde gebruiker kan ook niet meer inloggen. Alle adminroutes heb ik achter deze beveiliging gezet. Toen heb ik de data gekoppeld. Ik heb controllers gemaakt voor het dashboard, de gebruikers, de games en de API-status. In de UserController heb ik ingesteld wie wat mag aanpassen. Zo kan alleen de owner iemand admin maken. In de frontend heb ik de pagina /admin gemaakt met vier tabbladen. Als je geen admin bent, kom je niet op deze pagina. Ook heb ik een owner-account en twee admin-accounts gemaakt voor ons team. De wachtwoorden en keys staan in de .env en niet in de code. Toen ik het dashboard testte, kreeg ik een foutmelding. In het logbestand zag ik dat er een regel code miste. Die heb ik toegevoegd, en daarna werkte alles

xptracker-backend/routes/api.php

```php
Route::middleware(['auth:sanctum', 'admin'])->prefix('admin')->group(function () {
    Route::get('/dashboard', Admin\DashboardController::class);

    Route::get('/users', [Admin\UserController::class, 'index']);
    Route::patch('/users/{user}', [Admin\UserController::class, 'update']);
    Route::delete('/users/{user}', [Admin\UserController::class, 'destroy']);

    Route::get('/games', [Admin\GameController::class, 'index']);
    Route::post('/games', [Admin\GameController::class, 'store']);
    Route::patch('/games/{game}', [Admin\GameController::class, 'update']);
    Route::delete('/games/{game}', [Admin\GameController::class, 'destroy']);

    Route::get('/api-status', [Admin\ApiStatusController::class, 'index']);
    Route::post('/cache/clear', [Admin\ApiStatusController::class, 'clearCache']);
});
```

xptracker-backend/database/migrations/2026_10_23_132500_add_admin_fields_to_users_and_games.php

```php
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('role', 20)->default('user')->index()->after('email');
        $table->timestamp('banned_at')->nullable()->after('role');
    });

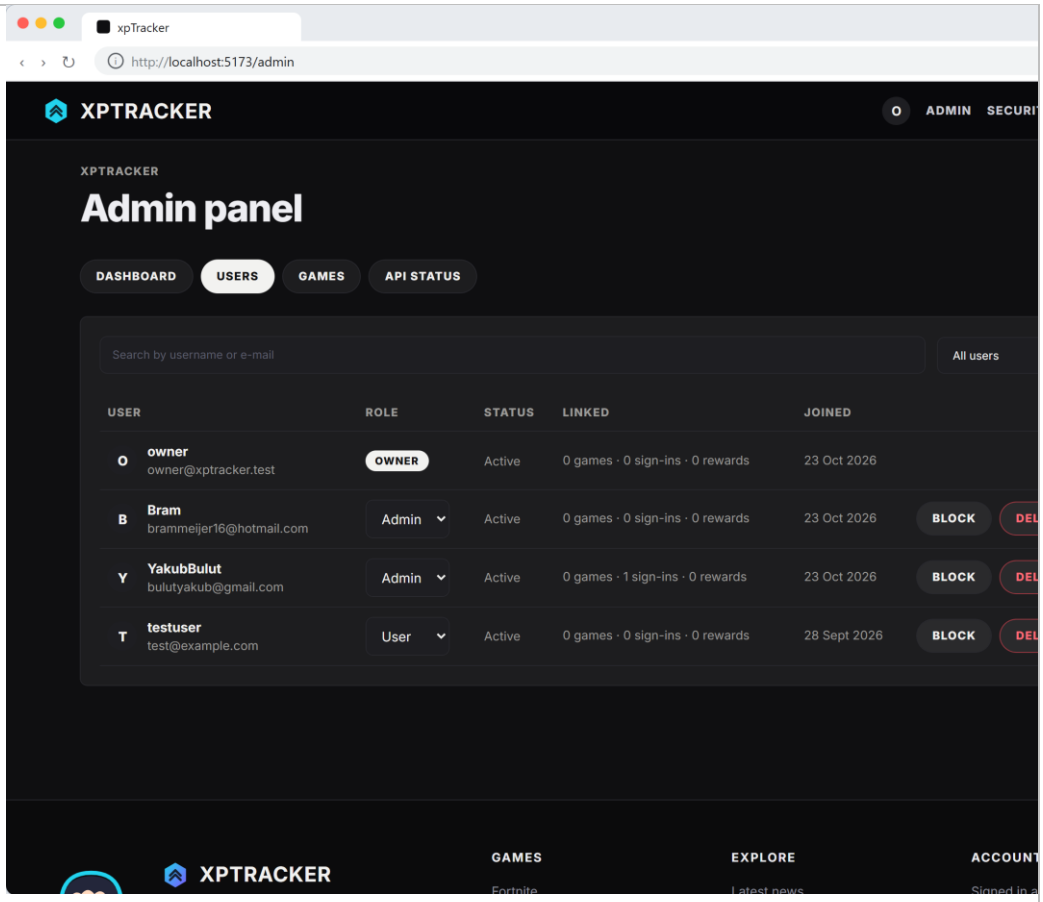
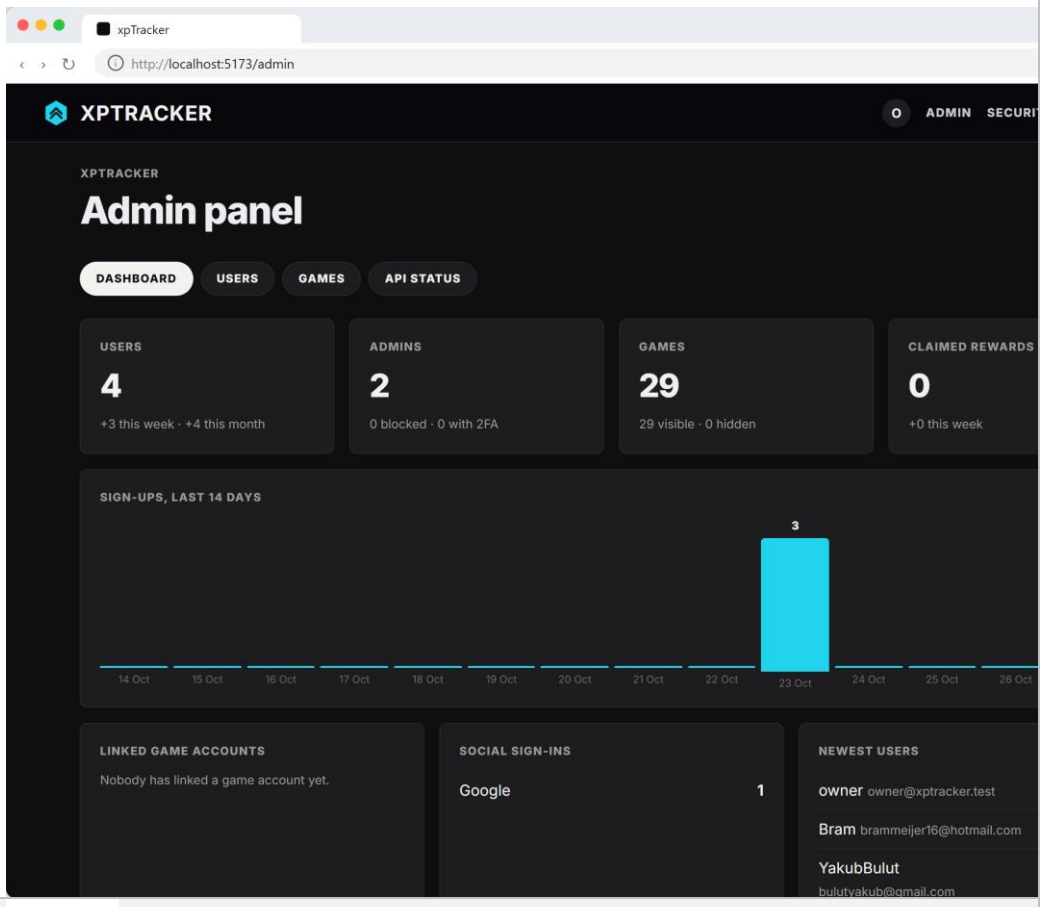
    Schema::table('games', function (Blueprint $table) {
        $table->boolean('is_visible')->default(true)->after('description');
    });
}
```

xptracker-backend/app/Http/Controllers/Admin/UserController.php

```php
private function canManage(User $actor, User $target): bool
{
    if ($actor->is($target) || $target->isOwner()) {
        return false;
    }

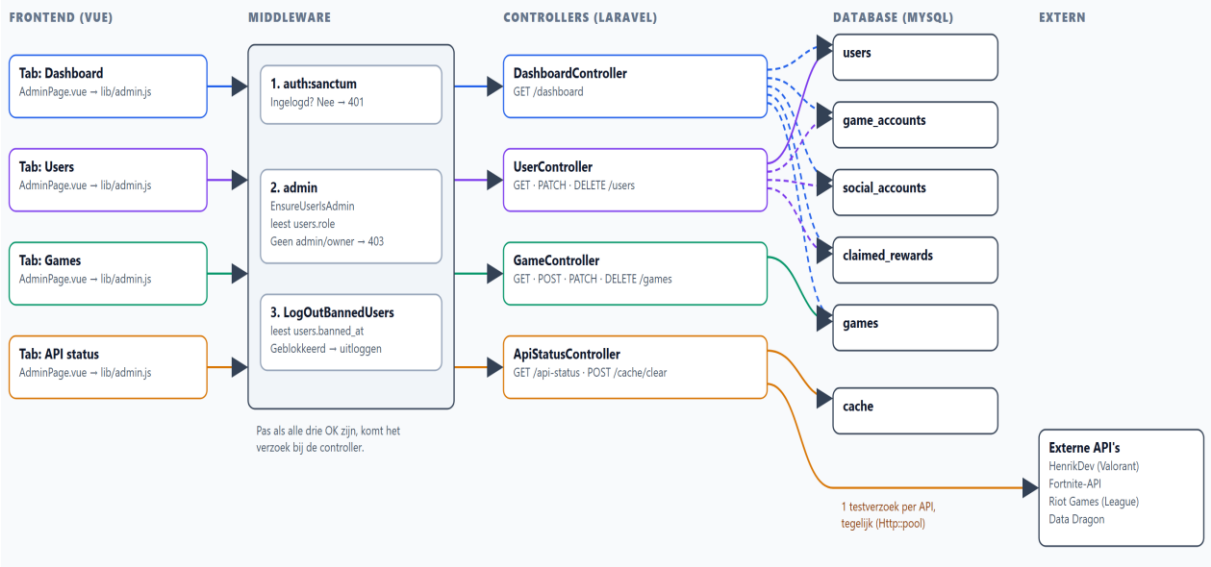
    return ! $target->isAdmin() || $actor->role->canManageAdmins();
}

private function refusal(User $actor, User $target): string
{
    return match (true) {
        $actor->is($target) => 'Je kunt je eigen account niet aanpassen vanuit het adminpaneel.',
        $target->isOwner() => 'Het owner-account kan niet worden aangepast.',
        default => 'Alleen de owner kan andere admins aanpassen.',
    };
}
```

	 
Resultaat	Ontwerp*

	Ontwerp Voordat ik ging bouwen, hebben we eerst bepaald wat de admin moest kunnen: gebruikers beheren, games beheren, een dashboard met cijfers en de status van de API's bekijken. Op basis daarvan heb ik het ontwerp gemaakt. In de database heb ik de kolommen role, banned_at en is_visible toegevoegd. Daarnaast heb ik drie rollen bedacht: user, admin en owner. Zo staat bij elke eis vast hoe die in de database, de backend en de frontend wordt opgelost. **Schematechnieken** Ik heb een Applicatiearchitectuur gemaakt om te laten zien welke tabellen de admin gebruikt en hoe ze aan elkaar gekoppeld zijn. Ook heb ik een schema gemaakt dat laat zien hoe het adminpaneel via de middleware en de controllers bij de data komt. Beide schema's heb ik onderaan dit Word-document in de bijlage gezet. **Onderbouwing** Ik heb gekozen voor één kolom role in plaats van losse kolommen zoals is_admin en is_owner. Zo kan een account nooit twee dingen tegelijk zijn. Voor de security zit de controle in de backend, met middleware. Alleen een knop verbergen in de frontend is niet veilig, want dan kan iemand de API nog steeds zelf aanroepen. Alleen de owner kan iemand admin maken, en niemand kan zichzelf aanpassen. Zo kun je jezelf niet buitensluiten. Voor de privacy staan wachtwoorden en API-keys in de .env en niet in de code, en die gaat niet naar GitHub. Het was ook haalbaar, omdat ik verder kon bouwen op wat er al was, zoals het inloggen en de beveiliging van routes. Daardoor kon ik het in één sprint afmaken.
Reflectie	**Feedback en feedforward** Van Thomas en Bram kreeg ik als feedback dat bijvoorbeeld: de beveiliging goed in de backend zit. Als tip kreeg ik bijvoorbeeld: eerder tests schrijven. **Kwaliteit** Ik vind het eindproduct goed gelukt. Het adminpaneel doet wat we hadden afgesproken en het is veilig. Het testen kan nog beter. **Tijd** Ik heb er ongeveer 2 dagen aan gewerkt in sprint 7. Het meeste werk zat in de beveiliging. **Proces** Het ging redelijk goed. Halverwege kwam er een owner-rol bij, waardoor ik de database moest aanpassen. Een fout in het dashboard heb ik via het logbestand opgelost. **Samenwerking** Met Thomas en Bram heb ik eerst besproken wat de admin moest kunnen. Ik werkte in een aparte branch, zodat ik hun werk niet in de weg zat.

Bijlage 1.1



Hierbij zie je de applicatiearchitectuur van de admin hoe het is verbonden aan elkaar

Bijlage 1.2